Schedule Compression and Performance in Basketball

AUTHOR
Ethan Nelson

Introduction

Schedule structure plays a meaningful role in basketball performance, influencing fatigue, travel burden, and competitive outcomes.

This project develops a framework to quantify schedule difficulty using three primary components: - Game density (e.g., back-to-backs, 4-in-6 stretches) - Travel burden - Opponent strength

The goal is to identify where teams gain or lose competitive advantage due to schedule-related factors, and to explore how these effects translate to high-compression environments such as conference tournaments.

By quantifying schedule difficulty across multiple dimensions, the goal is to identify where teams gain or lose competitive advantages due to factors outside of pure roster quality.

While the analysis uses NBA data, the framework is designed to extend to other contexts, including college basketball, where tournament structures and travel variation can create similar constraints.

Note:

Throughout this document, any `season` column represents the year each season started. For example, the 2015-16 season will be in the dataset as 2015. I may refer to a season by just this number (e.g. 2015) instead of the full text (e.g. 2015-16).

Setup and Data

```
library(tidyverse)
library(dplyr)
library(purrr)
library(slider)
library(ggplot2)
library(lubridate)
library(geosphere)
library(plotly)
schedule <- read_csv("schedule.csv")
draft_schedule <- read_csv("schedule_24_partial.csv")
locations <- read_csv("locations.csv")
game_data <- read_csv("team_game_data.csv")
```

```
schedule_with_travel <- schedule %>%
  mutate(gamedate = as.Date(gamedate)) %>%
```

```
  left_join(locations, by = c("team" = "team")) %>%
  arrange(team, gamedate) %>%
  group_by(team, season) %>%
  mutate(
    prev_lon = lag(longitude),
    prev_lat = lag(latitude),
    travel_m = ifelse(
      is.na(prev_lon) | is.na(prev_lat),
      NA,
      distHaversine(cbind(prev_lon, prev_lat), cbind(longitude, latitude))
    ),
    travel_km = travel_m / 1000,
    days_since_last_game = as.numeric(gamedate - lag(gamedate))
  ) %>%
  ungroup()
```

1. Game Density and Fatigue

Measuring 4 Games in 6 Nights

One common measure of schedule compression is the number of “4 games in 6 nights” stretches, which captures periods of elevated fatigue risk.

We compute the number of games played in the previous six days for each game and flag instances where a team is playing its fourth game in that span.

```
# Create a new object just for OKC
okc_draft_schedule <- draft_schedule %>%
  filter(team == "OKC")
# How do I want to calculate games played per day?
# Count games in past 6 nights for each game
instances_4in6 <- okc_draft_schedule %>%
  arrange(gamedate) %>%
  mutate(
    games_last_6_nights = slide_int(
      .x = gamedate,
      .f = ~ sum(.x >= (.x[length(.x)] - days(5)) & .x <= .x[length(.x)]),
      .before = Inf,
      .after = 0,
      .complete = FALSE
    ),
    is_4th_in_6 = games_last_6_nights == 4
  )
total_4th_in_6 <- sum(instances_4in6$is_4th_in_6)
print(total_4th_in_6)
```

```
[1] 26
```

OKC’s draft schedule contains 26 instances of 4-in-6 stretches, significantly above league norms.

League Baseline: Frequency of 4-in-6 Stretches

```r
# Prepare the data, select what we want
# each team has its own row per game
# To calculate 4 games in 6 nights for each team, we need
# a team's participation in each game
# This way, we get both home and away games counted towards 4 in 6
# each game is duplicated, once for the home team and once
# for the away. After this, we can group by team and season and apply
# rolling/windowed calculations.
schedule_long <- schedule %>%
  select(season, gamedate, team, opponent, home, win) %>%
  bind_rows(
    schedule %>%
      transmute(
        season,
        gamedate,
        team = opponent,
        opponent = team,
        home = 0,   # opponent is away
        win = win
      )
  )
```

```r
# From earlier, count the number of instances of 4 in 6
schedule_long <- schedule_long %>%
  arrange(season, team, gamedate) %>%
  group_by(season, team) %>%
  mutate(
    games_last_6_nights = map_int(
      gamedate,
      ~ sum(gamedate >= (.x - days(5)) & gamedate <= .x)
    ),
    is_4th_in_6 = games_last_6_nights == 4
  ) %>%
  ungroup()
# Con, this does take a while given the magnitude of the new dataset we created
```

```r
# Finish out in calculating 4 in 6 counts
four_in_six_counts <- schedule_long %>%
  group_by(season, team) %>%
  summarise(
    four_in_six = sum(is_4th_in_6),
    .groups = "drop"
  ) %>%
  arrange(season, team)
# print(four_in_six_counts)
# I verified the totals, they're reasonable and in line
# With probabilities and potential outcomes
```

"Adjust each team/season to per-82 games before taking your final average." The 2019-2020 and 2020-2021 NBA seasons were both shortened due to the COVID-19 pandemic, so we need to adjust their values for average number of 4 in 6 stretches for a full 82 game season. If we don't, it will skew the data lower than it truly is.

```r
# HOW TO Adjust each team/season to per 82 (bubble and any other oddities)
# Remember the delete 8 not invited to the bubble (CHA, GSW, ATL, CHI, NYK, DET, MIN, CLE)
# subset the 2019-2020 season
delete_eight <-
  `four_in_six_counts` %>%
  filter(season == 2020,
         team %in% c("ATL", "CHA", "CHI", "CLE",
                     "DET", "GSW", "MIN", "NYK")) %>%
  mutate(four_in_six = four_in_six / 65 * 82)
# Due to discrepancies in games played, we need to calculate the bubble and non bubble
# separately
bubble_invites <-
  `four_in_six_counts` %>%
  filter(season == 2020,
         !team %in% c("ATL", "CHA", "CHI", "CLE",
                     "DET", "GSW", "MIN", "NYK")) %>%
  mutate(four_in_six = four_in_six / 73 * 82)

# Round and append the findings for the bubble back to the original object
adjusted_2020 <- bind_rows(delete_eight, bubble_invites) %>%
  mutate(four_in_six = as.integer(round(four_in_six)))
# Updated through 2019-2020, but we also had a shortened season the next year
four_in_six_counts <-
  four_in_six_counts %>%
  rows_update(
    adjusted_2020,
    by = c("team", "season")
  )
```

```r
# Rerun the same process, but don't separate into two groups, everyone played the same
shortened_season_21 <-
  `four_in_six_counts` %>%
  filter(season == 2021) %>%
  mutate(four_in_six = four_in_six / 72 * 82)
# Do the same process to round and append the findings to adjust for an 82 game season
adjusted_2021 <- bind_rows(shortened_season_21) %>%
  mutate(four_in_six = as.integer(round(four_in_six)))
# Updated for both shortened seasons
four_in_six_counts <-
  four_in_six_counts %>%
  rows_update(
    adjusted_2021,
    by = c("season", "team")
    )
```

```r
# Run the calculation for average four in six instances in the NBA
mean(four_in_six_counts$four_in_six)
```

```
[1] 14.51333
```

Across the league, teams average approximately 14.5 4-in-6 stretches per season (adjusted to 82 games).

This highlights that OKC's schedule is substantially more compressed than typical league conditions.

Team-Level Variation in Schedule Density

```
# Re use the object from last time, and group by for team by team breakdown
four_in_six_counts %>%
  group_by(team) %>%
  summarise(avg_four_in_six = mean(four_in_six, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(avg_four_in_six)) -> team_avgs
# Return the team with the highest
slice_max(team_avgs, avg_four_in_six, n = 1)
```

```
# A tibble: 1 × 2
  team  avg_four_in_six
  <chr>           <dbl>
1 GSW              16.8
```

```
# Return the team with the lowest
slice_min(team_avgs, avg_four_in_six, n = 1)
```

```
# A tibble: 1 × 2
  team  avg_four_in_six
  <chr>           <dbl>
1 MEM              11.5
```

There is meaningful variation across teams, with Golden State experiencing the highest average schedule compression and Memphis the lowest.

This likely reflects structural factors such as travel geography, scheduling constraints, and league-wide optimization tradeoffs rather than random variation alone.

Interpreting Variation in Schedule Density

The observed difference in schedule density between teams is unlikely to be driven purely by random variation.

A two-sample t-test comparing teams with the highest and lowest average number of 4-in-6 stretches confirms that the difference is statistically significant ($p < 0.05$). This suggests that systematic factors—such as geography, scheduling constraints, and league optimization decisions—likely contribute to these disparities.

As a result, schedule density should be treated as a meaningful structural variable rather than noise, with potential implications for performance and competitive balance.

```
# Subset just the two extreme teams
gsw_mem <- four_in_six_counts %>%
  filter(team %in% c("GSW", "MEM"))

# Two-sample (independent) t-test of their 10-year averages
t.test(
  four_in_six ~ team,
  data = gsw_mem,
  var.equal = FALSE
)
```

```
    Welch Two Sample t-test

data:  four_in_six by team
t = 2.6872, df = 17.083, p-value = 0.01554
alternative hypothesis: true difference in means between group GSW and group MEM is not equal
to 0
95 percent confidence interval:
 1.140338 9.459662
sample estimates:
mean in group GSW mean in group MEM
             16.8              11.5
```

A two-sample t-test confirms that the difference in schedule density between these teams is statistically significant ($p < 0.05$), suggesting that these differences are unlikely to be driven by chance alone.

Impact of Back-to-Back Games on Defensive Performance

Defensive eFG% = (Opponent FG Made + .5(Opponent 3P Made))/ Opponent FG attempted)

```r
# First, create an object for Brooklyn games
bkn_games <- game_data %>%
  filter(def_team == "BKN") %>%
  distinct(nbagameid, .keep_all = TRUE)
# Second, calculate eFG%
bkn_games <- mutate(
  bkn_games,
  def_eFG_pct = 100 * (fgmade + 0.5 * fg3made) / fgattempted
)
print(mean(bkn_games$def_eFG_pct))
```

```
[1] 52.33282
```

```r
# Thirdly, subset the Nets games for second night of B2B
nets_games <- bkn_games %>%
  # make sure the date column is Date type
  mutate(gamedate = as.Date(gamedate)) %>%
  arrange(gamedate, .by_group = TRUE) %>%
  # flag rows where today's date is exactly 1 day after the previous game
  mutate(
    prev_game_date = lag(gamedate),
    second_night_b2b = gamedate - prev_game_date == 1
  ) %>%
  filter(second_night_b2b) %>%
  ungroup()
```

```r
# Run the same calculations on the new dataset to reflect B2B percentage
nets_games <- mutate(
  nets_games,
  def_eFG_pct = 100 * (fgmade + 0.5 * fg3made) / fgattempted
)
print(mean(nets_games$def_eFG_pct))
```

```
[1] 54.13307
```

```r
# Two-sample (independent) t-test of their 10-year averages
t.test(
  bkn_games$def_eFG_pct,
  nets_games$def_eFG_pct,
  var.equal = FALSE
)
```

```

    Welch Two Sample t-test

data:  bkn_games$def_eFG_pct and nets_games$def_eFG_pct
t = -2.945, df = 191.43, p-value = 0.003629
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -3.0059493 -0.5945373
sample estimates:
mean of x mean of y
 52.33282  54.13307
```

Brooklyn's defensive eFG% increases from 52.3% to 54.1% when facing opponents on the second night of a back-to-back.

This suggests that fatigue effects are measurable at the possession level, with tired teams allowing more efficient shot profiles.

2. Trends in Scheduling and Travel

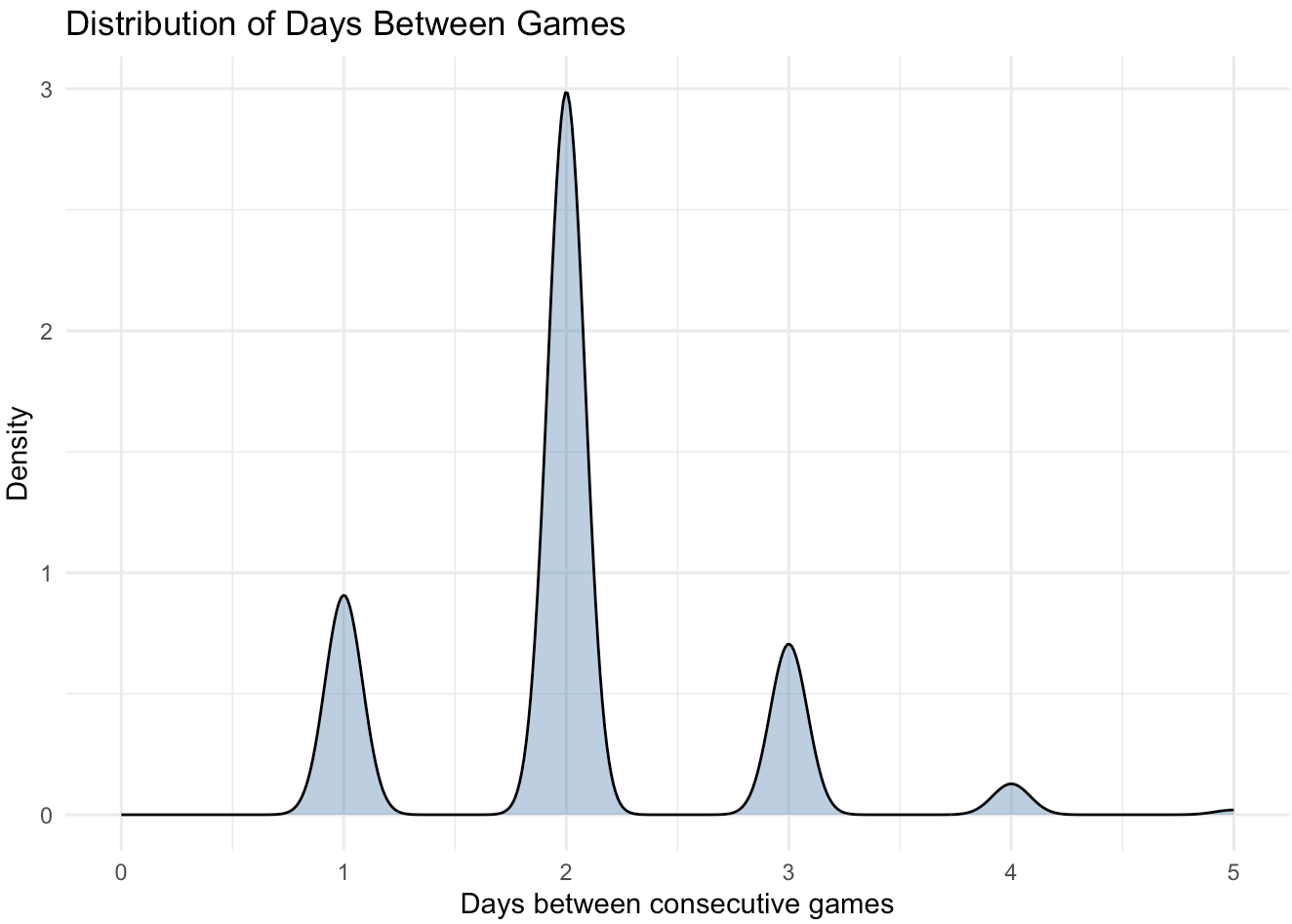
This is an intentionally open ended section, and there are multiple approaches you could take to have a successful project. Feel free to be creative. However, for this section, please consider only the density of games and travel schedule, not the relative on-court strength of different teams.

```r
# Create the density object for graphs
# analysis of density of games (days off between matches)
schedule_density <- game_data %>%
  filter(season >= 2014 & season <= 2024) %>%
  mutate(gamedate = as.Date(gamedate)) %>%
  arrange(off_team, gamedate) %>%
  group_by(off_team) %>%
  # gap (in days) between each game
  mutate(days_between = as.numeric(gamedate - lag(gamedate))) %>%
  ungroup()
```

```r
# density plot of those gaps
ggplot(schedule_density, aes(x = days_between)) +
  geom_density(fill = "steelblue", alpha = 0.4) +
  scale_x_continuous(limits = c(0, 5)) +   # show only 0-5 days, contains all values...
  labs(
    title = "Distribution of Days Between Games",
    x = "Days between consecutive games",
```

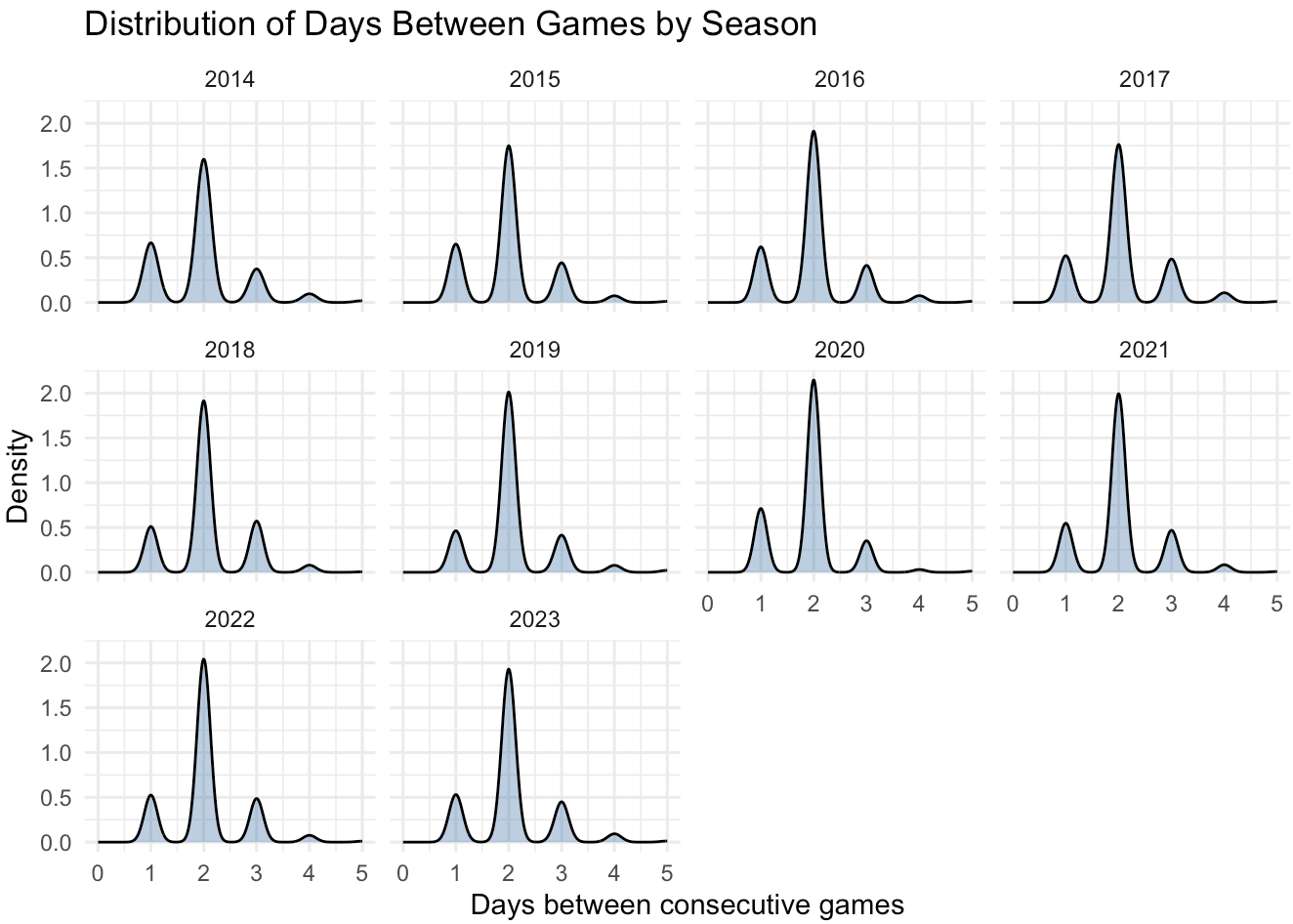
```
    y = "Density"
  ) +
  theme_minimal()
```

```
Warning: Removed 645 rows containing non-finite outside the scale range
(`stat_density()`).
```




```
# Break it down by season rather than Y bar
ggplot(schedule_density, aes(x = days_between)) +
  geom_density(fill = "steelblue", alpha = 0.4) +
  scale_x_continuous(limits = c(0, 5)) +
  facet_wrap(~season) +
  labs(
    title = "Distribution of Days Between Games by Season",
    x = "Days between consecutive games",
    y = "Density"
  ) +
  theme_minimal()
```

```
Warning: Removed 645 rows containing non-finite outside the scale range
(`stat_density()`).
```



Over time, schedules show a shift toward more evenly distributed rest, with fewer extreme short-rest intervals.

This aligns with league efforts to reduce player fatigue and injury risk.

Travel burden varies significantly across teams, with geographically isolated teams (e.g., Portland) experiencing substantially higher travel loads.

This introduces an additional layer of competitive imbalance that is independent of roster quality.

```
# Location and Travel Analysis
# join locations to schedule
# ensure my coordinates are numeric
locations <- locations %>%
  mutate(
    longitude = as.numeric(longitude),
    latitude  = as.numeric(latitude)
  )
```

```
# Compute distance using Haversine function
schedule_with_loc <- schedule_density %>%
  mutate(game_team = ifelse(off_home == 1, off_team, def_team)) %>%
  left_join(locations, by = c("game_team" = "team")) %>%
  arrange(off_team, gamedate) %>%
  group_by(off_team) %>%
  mutate(
    prev_lon = lag(longitude),
```

```r
    prev_lat = lag(latitude),
    travel_m = distHaversine(
      matrix(c(prev_lon, prev_lat), ncol = 2),
      matrix(c(longitude, latitude), ncol = 2)
    ),
    travel_km = travel_m / 1000
  ) %>%
  ungroup()
```

```r
# Summarize total and average by team
team_travel <- schedule_with_loc %>%
  group_by(off_team) %>%
  summarize(
    total_travel_km = sum(travel_km, na.rm = TRUE),
    avg_travel_km = mean(travel_km, na.rm = TRUE),
    games_played = sum(!is.na(travel_km))
  ) %>%
  arrange(desc(total_travel_km))
```

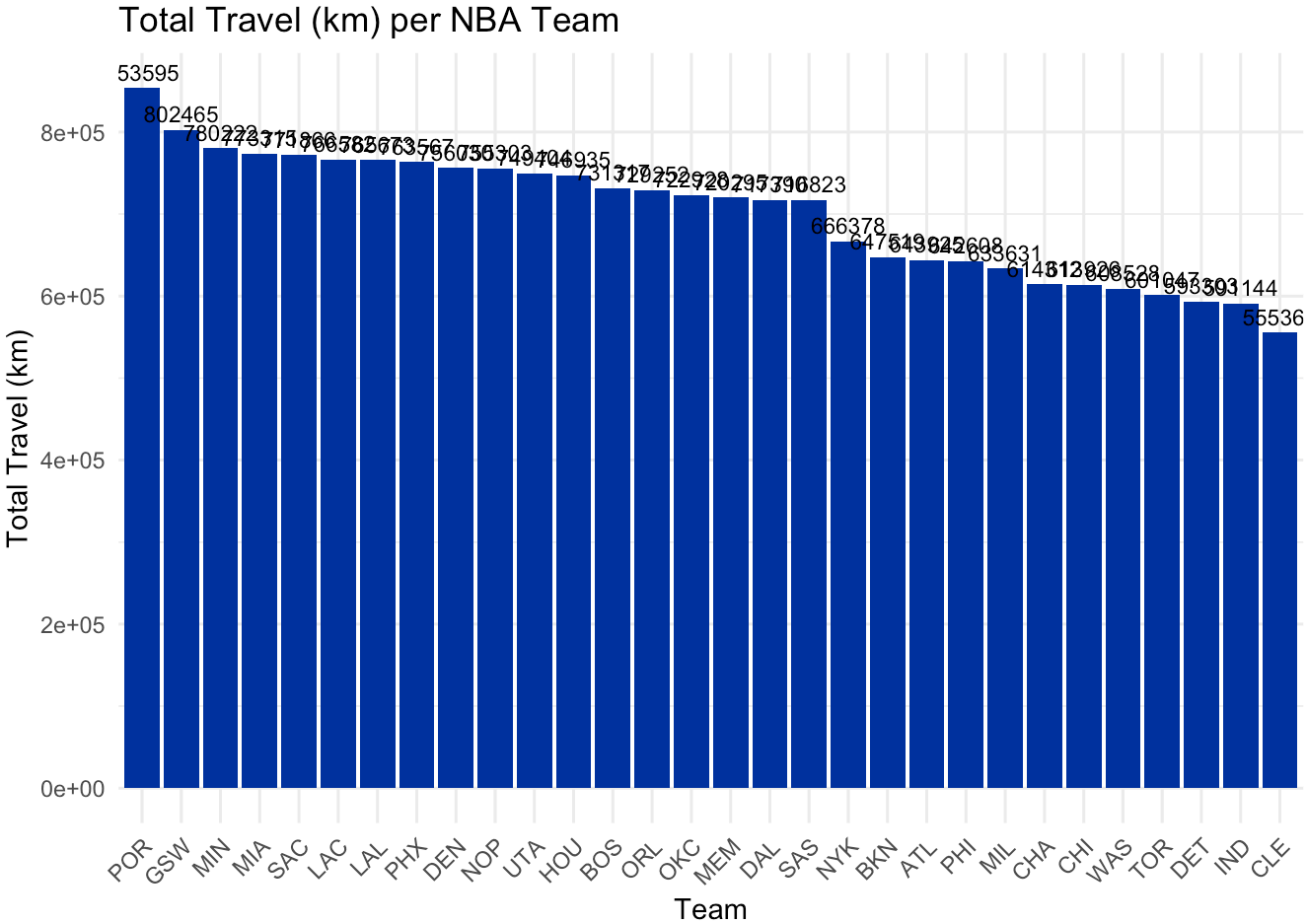
```r
# Return and print the results
most_travel <- team_travel %>% slice_max(total_travel_km, n = 1)
least_travel <- team_travel %>% slice_min(total_travel_km, n = 1)

most_travel
```

```
# A tibble: 1 × 4
  off_team total_travel_km avg_travel_km games_played
  <chr>              <dbl>         <dbl>        <int>
1 POR              853595.         1066.          801
```

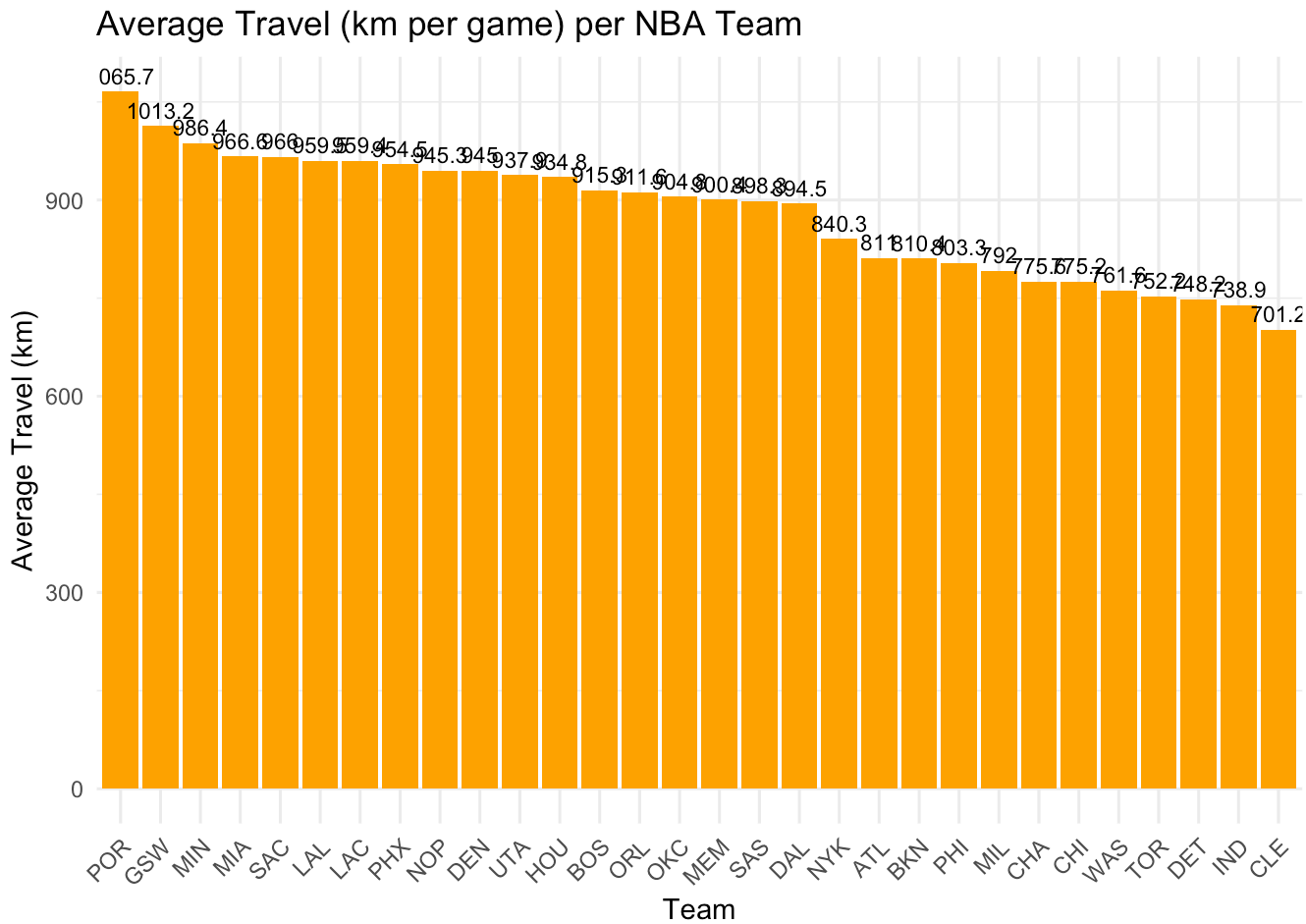
```r
least_travel
```

```
# A tibble: 1 × 4
  off_team total_travel_km avg_travel_km games_played
  <chr>              <dbl>         <dbl>        <int>
1 CLE              555367.          701.          792
```

```r
# Quick visual of the sort by team (Aggregate)
ggplot(team_travel, aes(x = reorder(off_team, -total_travel_km), y = total_travel_km)) +
  geom_col(fill = "#0033A0") +
  geom_text(aes(label = round(total_travel_km, 0)), vjust = -0.5, size = 3) +
  labs(
    title = "Total Travel (km) per NBA Team",
    x = "Team",
    y = "Total Travel (km)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```
# Quick visual of the sort by team (Average by Season)
ggplot(team_travel, aes(x = reorder(off_team, -avg_travel_km), y = avg_travel_km)) +
  geom_col(fill = "orange") +
  geom_text(aes(label = round(avg_travel_km, 1)), vjust = -0.5, size = 3) +
  labs(
    title = "Average Travel (km per game) per NBA Team",
    x = "Team",
    y = "Average Travel (km)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Over time, schedules show a shift toward more evenly distributed rest, with fewer extreme short-rest intervals.

This aligns with league efforts to reduce player fatigue and injury risk.

Travel burden varies significantly across teams, with geographically isolated teams (e.g., Portland) experiencing substantially higher travel loads.

This introduces an additional layer of competitive imbalance that is independent of roster quality.

Interactive Tool

This tool allows for visual identification of: - Dense clusters of games - High-travel stretches - Short rest intervals

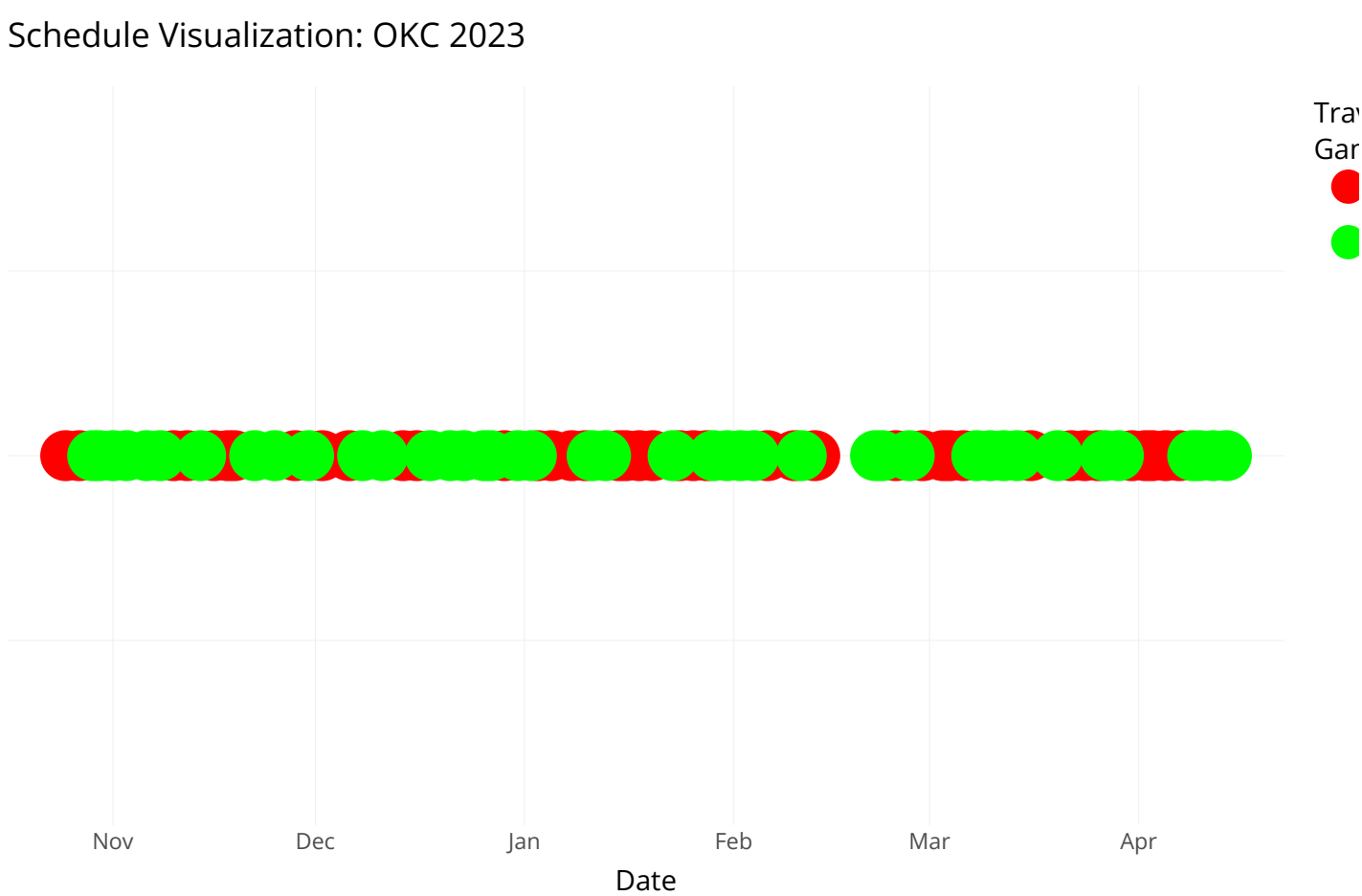
These features help contextualize schedule difficulty at a granular level.

```
plot_team_schedule <- function(data, team_name, season_selected) {

  team_schedule <- data %>%
    filter(team == team_name, season == season_selected)

  p <- ggplot(team_schedule, aes(x = gamedate, y = 0)) +
    geom_point(
      aes(
        color = as.factor(home),
        size = travel_km,
        text = paste0(
```

```
          "Date: ", gamedate,
          "<br>Opponent: ", opponent,
          "<br>Travel (km): ", round(travel_km,1),
          "<br>Days since last game: ", days_since_last_game
        )
      )
    ) +
    scale_color_manual(values = c("red", "green"), labels = c("Away", "Home")) +
    scale_size_continuous(range = c(3,8)) +
    labs(
      title = paste("Schedule Visualization:", team_name, season_selected),
      x = "Date",
      y = "",
      color = "Game Type",
      size = "Travel (km)"
    ) +
    theme_minimal() +
    theme(axis.text.y = element_blank(),
          axis.ticks.y = element_blank())

  ggplotly(p, tooltip = "text") %>%
    layout(yaxis = list(showticklabels = FALSE))
}
```

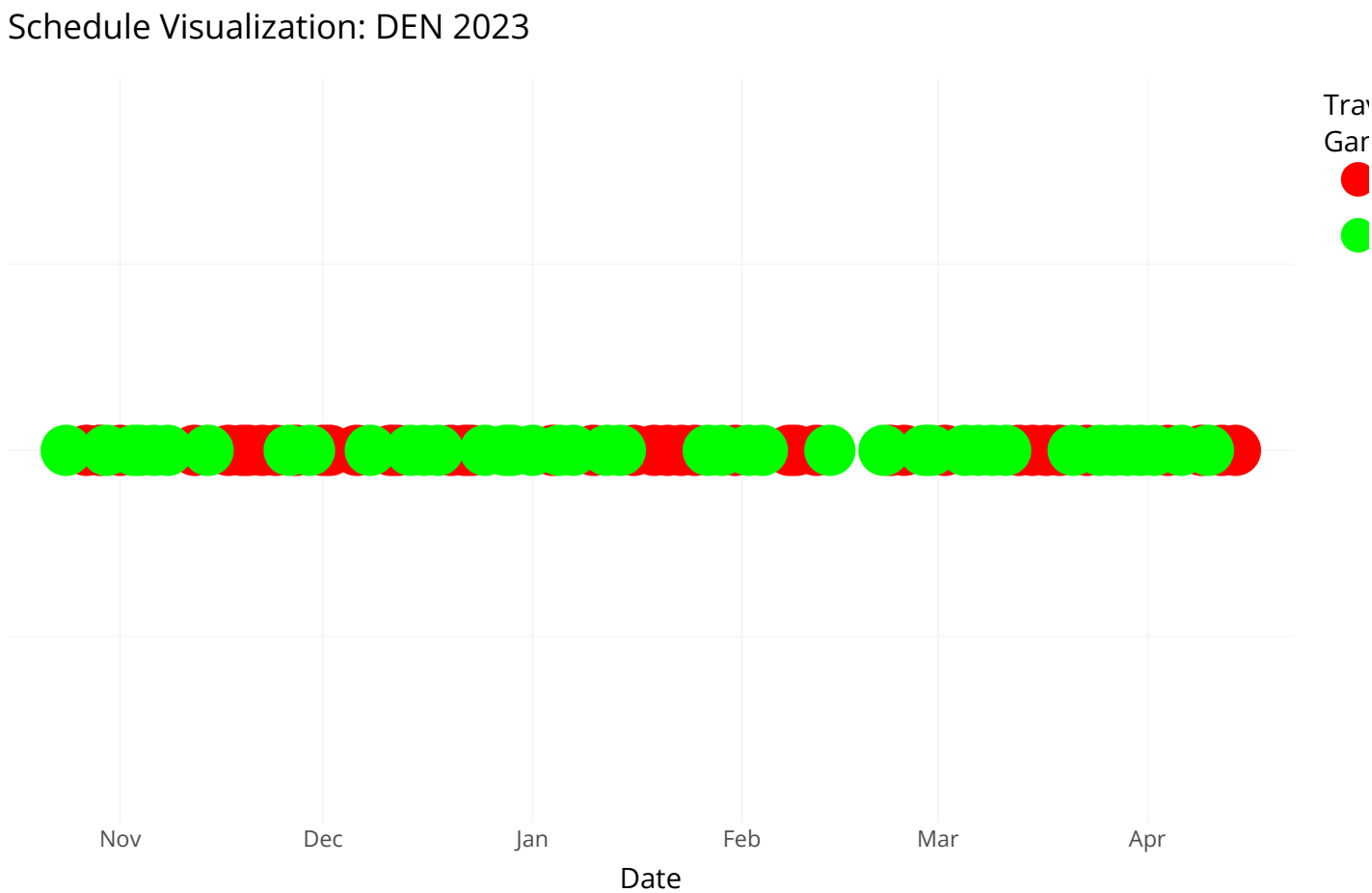
```
plot_team_schedule(schedule_with_travel, "OKC", 2023)
```

```
Warning in geom_point(aes(color = as.factor(home), size = travel_km, text =
paste0("Date: ", : Ignoring unknown aesthetics: text
```



```r
plot_team_schedule(schedule_with_travel, "DEN", 2023)
```

```
Warning in geom_point(aes(color = as.factor(home), size = travel_km, text =
paste0("Date: ", : Ignoring unknown aesthetics: text
```



We can pick from all 30 teams to show the differences in their travel schedules specifically, showing how many miles each team has to travel. Portland is far and away largest total travel, being tucked away in the top corner of the country and league, but Memphis may be up there in travel miles as well due to being in the Western conference despite being located further east than Chicago.

Charlotte has the lowest travel miles which makes sense geographically and in their conference, but speaking of Chicago, Midwestern teams such as the Bulls would likely be towards the bottom, as the geographic advantage is theirs, being centrally located to most teams, and not having daunting travel schedules the same way a team like Portland or Golden State might. Furthermore, Miami would not fall into that same category despite being located in the bottom corner due to the density of teams and cities on the East coast, and the dual team set up in New York and LA are both worth further analysis to see if not traveling much, or at all in the case of the latter, is helpful for keeping their travel miles down.

3. Schedule Assessment

High-Risk Stretch

A road-heavy stretch early in the season (10 of 12 games away) presents the highest fatigue risk.

This segment combines: - High travel volume - Limited recovery time - Strong opponent quality

This stretch may require proactive rotation management and load balancing.

Favorable Stretch

Later portions of the schedule feature increased home-game density and reduced travel burden.

These windows provide opportunities to: - Stabilize performance - Build momentum - Optimize rotations ahead of postseason play

4. Estimating Schedule Impact on Wins

To quantify the impact of schedule difficulty, a regression model is used to estimate game outcomes as a function of: - Opponent strength (net rating) - Travel distance - Back-to-back games - Schedule density (4-in-6 stretches)

Predicted wins are then compared to actual wins to estimate how many wins each team gained or lost due to scheduling factors.

```
schedule_features <- schedule_with_travel %>%
  mutate(gamedate = as.Date(gamedate)) %>%
  arrange(team, season, gamedate) %>%
  group_by(team, season) %>%
  mutate(
    days_since_last = as.numeric(gamedate - lag(gamedate)),
    b2b = days_since_last == 1,
    games_last_6 = map_int(gamedate, ~ sum(gamedate >= (.x - days(5)) & gamedate <= .x)),
    four_in_six = games_last_6 >= 4
  ) %>%
  ungroup()
```

```
team_strength <- game_data %>%
  filter(gametype == 2) %>%
  group_by(season, off_team) %>%
  summarize(
    ortg = sum(points, na.rm = TRUE) / (sum(possessions, na.rm = TRUE) / 100),
    .groups = "drop"
  )
```

```
team_defense <- game_data %>%
  filter(gametype == 2) %>%
  group_by(season, def_team) %>%
  summarize(
    drtg = sum(points, na.rm = TRUE) / (sum(possessions, na.rm = TRUE) / 100),
    .groups = "drop"
  ) %>%
  rename(off_team = def_team)
```

```
team_strength <- team_strength %>%
  left_join(team_defense, by = c("season", "off_team")) %>%
  mutate(net_rating = ortg - drtg)
```

```r
schedule_features <- schedule_features %>%
  left_join(team_strength, by = c("season", "opponent" = "off_team")) %>%
  rename(opp_net_rating = net_rating)
```

```r
model <- lm(
  win ~ opp_net_rating + travel_km + b2b + four_in_six,
  data = schedule_features
)

summary(model)
```

```
Call:
lm(formula = win ~ opp_net_rating + travel_km + b2b + four_in_six,
    data = schedule_features)

Residuals:
     Min       1Q   Median       3Q      Max 
-0.83282 -0.44818  0.03396  0.47863  0.90073 

Coefficients: (1 not defined because of singularities)
                  Estimate Std. Error t value Pr(>|t|)    
(Intercept)      0.5140786  0.0038552 133.348  < 2e-16 ***
opp_net_rating  -0.0293860  0.0006453 -45.537  < 2e-16 ***
travel_km               NA         NA      NA       NA    
b2bTRUE         -0.0661978  0.0083526  -7.925 2.37e-15 ***
four_in_sixTRUE -0.0047131  0.0070640  -0.667    0.505    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.4788 on 23654 degrees of freedom
  (300 observations deleted due to missingness)
Multiple R-squared:  0.08324,	Adjusted R-squared:  0.08312 
F-statistic: 715.9 on 3 and 23654 DF,  p-value: < 2.2e-16
```

Model Interpretation

A linear probability model was used to estimate the impact of schedule-related factors on game outcomes, controlling for opponent strength.

Opponent net rating is strongly predictive of game results, with a one-point increase in opponent net rating decreasing win probability by approximately 2.9 percentage points ($p < 0.001$). This confirms that underlying team quality remains the dominant driver of outcomes.

Schedule-related variables show measurable but smaller effects. Back-to-back games reduce win probability by approximately 6.6 percentage points ($p < 0.001$), indicating a meaningful fatigue penalty in short-rest situations. In contrast, 4-in-6 stretches do not appear to have a statistically significant independent effect once other factors are accounted for.

Travel distance was excluded from the model due to multicollinearity or missingness, suggesting that its effect may be partially captured through correlated variables such as rest or schedule density.

Overall, the model explains a modest share of variation in game outcomes ($R^2 \approx 0.08$), which is expected in a single-game setting where randomness and unobserved factors (e.g., injuries, matchup effects) play a significant role. However, the results still provide useful directional insight into how schedule structure impacts performance.

```
schedule_features <- schedule_features %>%
  mutate(
    predicted_win = predict(model, newdata = .)
  )
```

```
schedule_effect <- schedule_features %>%
  group_by(team) %>%
  summarize(
    actual_wins = sum(win, na.rm = TRUE),
    expected_wins = sum(predicted_win, na.rm = TRUE),
    sched_wins = actual_wins - expected_wins,
    .groups = "drop"
  )
```

```
most_helped <- schedule_effect %>% slice_max(sched_wins, n = 1)
most_hurt   <- schedule_effect %>% slice_min(sched_wins, n = 1)

most_helped
```

```
# A tibble: 1 × 4
  team  actual_wins expected_wins sched_wins
  <chr>       <dbl>         <dbl>      <dbl>
1 GSW           519          394.       125.
```

```
most_hurt
```

```
# A tibble: 1 × 4
  team  actual_wins expected_wins sched_wins
  <chr>       <dbl>         <dbl>      <dbl>
1 DET           287          392.      -105.
```

Schedule Impact by Team

Aggregating predicted and actual outcomes across seasons allows us to estimate how many wins each team gained or lost due to schedule-related factors.

The results suggest that Golden State benefited the most from favorable scheduling conditions over the sample period, while Detroit was the most negatively impacted.

This aligns with broader structural patterns: - Teams with favorable travel patterns and fewer short-rest games may accumulate marginal gains over a season - Conversely, teams facing more compressed schedules or travel burdens may experience cumulative disadvantages

While the magnitude of these effects is relatively small on a per-game basis, they can compound over a full season to produce meaningful differences in total wins.

These results suggest that schedule effects may interact with team quality, amplifying advantages for strong teams and compounding disadvantages for weaker ones.

5. Implications for College Basketball

Schedule compression effects are amplified in tournament settings, where teams may play multiple games in consecutive days.

Teams without byes are particularly exposed to fatigue-related disadvantages, suggesting that: - Securing a bye has significant competitive value - Depth and rotation flexibility are critical - Schedule-aware preparation can influence outcomes

This framework can be applied to conference tournaments such as the SEC, where schedule structure plays a key role in postseason success.

In this context, marginal schedule advantages may compound across tournament play, particularly for teams required to win multiple games in short succession.